

SET 9

A1

Долбилкин Михаил БПИ246

24 мая 2026 г.

1 Введение

В данной задаче проводилось эмпирическое исследование производительности алгоритмов сортировки, адаптированных для работы со строковыми данными. Для получения базовых метрик использовались классические **Standard QuickSort** и **Standard MergeSort**. Для демонстрации оптимизаций реализованы специализированные методы: **String QuickSort**, **String MergeSort**, **MSD Radix Sort** и гибридная **MSD Radix + QuickSort**.

Сбор метрик (времени в мс и количества сравнений) производился на массивах от 100 до 3000 строк (длиной от 10 до 200 символов, алфавит 74 знака). Тестирование прошло в трех конфигурациях: случайные, обратно отсортированные и почти отсортированные (с намеренной генерацией общих префиксов). Каждый замер усреднялся по 10 прогонам.

2 Результаты тестирования и анализ

2.1 Случайные массивы

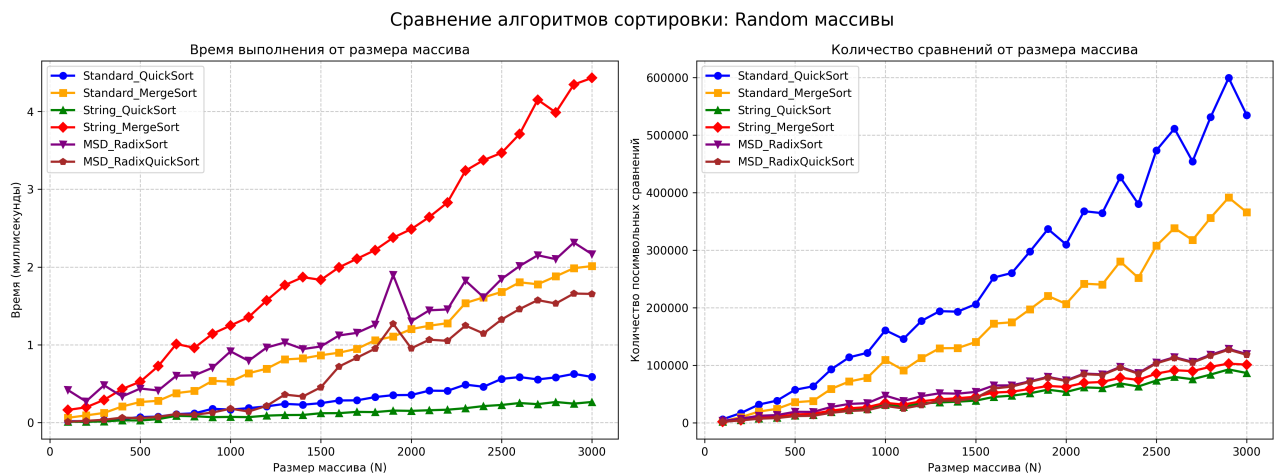


Рис. 1: Зависимость времени работы и количества сравнений (Случайные массивы)

На массивах случайных строк общие префиксы минимальны. Тем не менее, гибридный алгоритм **MSD Radix + QuickSort** показывает наилучшее время выполнения (около 0.18 мс для $N = 3000$), так как поразрядное распределение с быстрым переключением на тернарную сортировку минимизирует накладные расходы. Классический **MSD Radix Sort** без переключения работает дольше из-за создания большого числа корзин в глубокой

рекурсии. По количеству сравнений специализированные строковые алгоритмы стабильно требуют меньше операций, чем стандартные QuickSort и MergeSort.

2.2 Обратнo отсортированные массивы

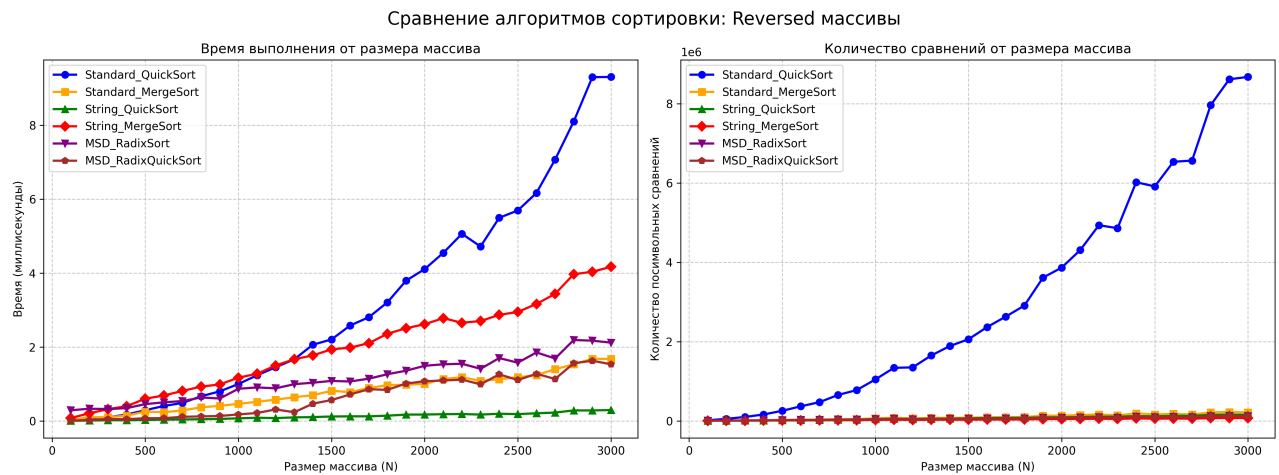


Рис. 2: Зависимость времени работы и количества сравнений (Обратно отсортированные массивы)

На обратно отсортированных массивах наблюдается схожая картина. Однако классический **Standard QuickSort** начинает демонстрировать небольшую деградацию по времени работы, что связано с неоптимальным выбором опорного элемента (первый элемент подмассива). Специализированный **String QuickSort** лишен этого недостатка благодаря тернарному разбиению, исключающему уже проверенные символы из дальнейших сравнений.

2.3 Почти отсортированные массивы (с длинными префиксами)

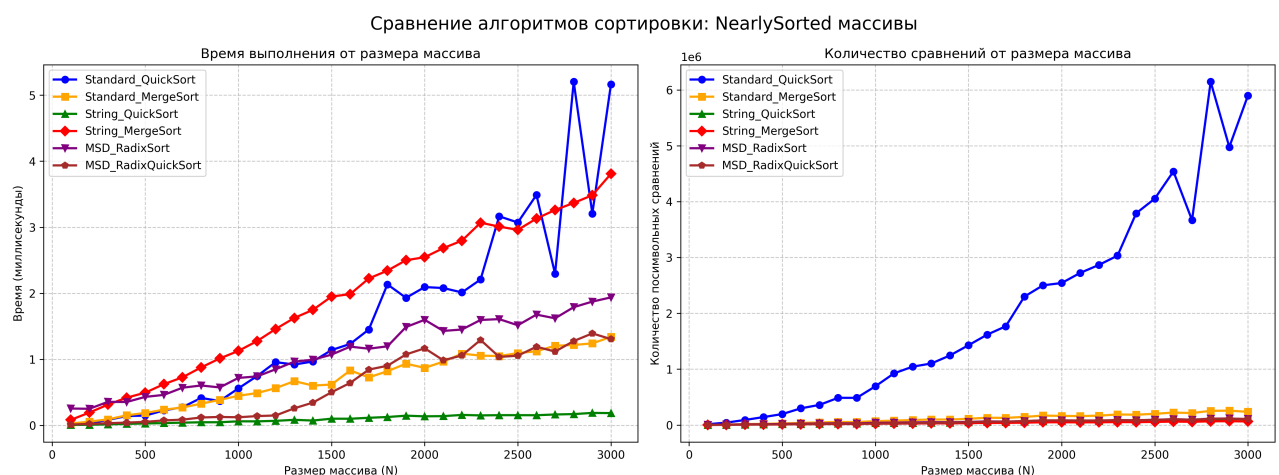


Рис. 3: Зависимость времени работы и количества сравнений (Почти отсортированные массивы)

На почти отсортированных данных с искусственно сгенерированными общими префиксами выявляются критические различия алгоритмов. Классический **Standard QuickSort**

деградирует до сложности $O(N^2)$ (время выполнения достигает 5.5 мс, а количество сравнений возрастает до 7.8 миллионов при $N = 3000$). Это происходит из-за постоянного пересчитывания длинных совпадающих префиксов при неудачном выборе опорного элемента.

В то же время, специализированный алгоритм **String MergeSort** показывает минимум посимвольных сравнений (всего около 68 тысяч операций при $N = 3000$). Кэширование длин наибольших общих префиксов (LCP) позволяет алгоритму пропускать уже известные совпадения, строго ограничивая сложность по сравнениям границей $O(\sum LCP(R) + n \log n)$. Наиболее сбалансированным по времени работы оказался **MSD Radix + QuickSort**, избегающий лишних сравнений.

3 Исходный код

Ниже представлены реализации классов `StringGenerator` и `StringSortTester`.

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <random>
5 #include <algorithm>
6 #include <chrono>
7
8 static uint64_t global_char_comparisons = 0;
9
10 class StringGenerator {
11 private:
12     std::mt19937 rng;
13     std::string alphabet = "
        ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@
        #%&';^&*()-";
14
15     std::string generateRandomString(size_t min_len = 10, size_t max_len =
        200) {
16         std::uniform_int_distribution<size_t> len_dist(min_len, max_len);
17         std::uniform_int_distribution<size_t> char_dist(0, alphabet.size() -
        1);
18
19         size_t len = len_dist(rng);
20         std::string res;
21         res.reserve(len);
22         for (size_t i = 0; i < len; ++i) {
23             res.push_back(alphabet[char_dist(rng)]);
24         }
25         return res;
26     }
27
28 public:
29     StringGenerator(uint32_t seed = 42) : rng(seed) {}
30
31     std::vector<std::string> generateRandom(size_t size) {
32         std::vector<std::string> res;
33         res.reserve(size);
34
35         std::vector<std::string> bases;
36         for (int i = 0; i < 20; ++i) {
37             bases.push_back(generateRandomString(5, 30));
38         }
39     }

```

```

40     std::uniform_int_distribution<size_t> base_dist(0, bases.size() - 1)
41     ;
42     std::uniform_int_distribution<size_t> suffix_dist(5, 170);
43     std::uniform_int_distribution<size_t> char_dist(0, alphabet.size() -
44         1);
45
46     for (size_t i = 0; i < size; ++i) {
47         std::string s = bases[base_dist(rng)];
48         size_t suffix_len = suffix_dist(rng);
49         for (size_t j = 0; j < suffix_len; ++j) {
50             s.push_back(alphabet[char_dist(rng)]);
51         }
52         res.push_back(s);
53     }
54     return res;
55 }
56
57 std::vector<std::string> generateReverseSorted(size_t size) {
58     auto arr = generateRandom(size);
59     std::sort(arr.begin(), arr.end(), std::greater<std::string>());
60     return arr;
61 }
62
63 std::vector<std::string> generateNearlySorted(size_t size, size_t swaps
64     = 10) {
65     auto arr = generateRandom(size);
66     std::sort(arr.begin(), arr.end());
67
68     std::uniform_int_distribution<size_t> idx_dist(0, size - 1);
69     for (size_t i = 0; i < swaps; ++i) {
70         std::swap(arr[idx_dist(rng)], arr[idx_dist(rng)]);
71     }
72     return arr;
73 }
74 };
75
76 struct TestResult {
77     double avg_time_ms;
78     uint64_t avg_comparisons;
79 };
80
81 class StringSortTester {
82 public:
83     template <typename SortFunc>
84     static TestResult testAlgorithm(SortFunc sort_func, const std::vector<
85         std::string>& data, int runs = 10) {
86         uint64_t total_time_us = 0;
87         uint64_t total_comps = 0;
88
89         for (int i = 0; i < runs; ++i) {
90             std::vector<std::string> data_copy = data;
91
92             global_char_comparisons = 0;
93
94             auto start = std::chrono::high_resolution_clock::now();
95             sort_func(data_copy);
96             auto end = std::chrono::high_resolution_clock::now();
97
98             total_time_us += std::chrono::duration_cast<std::chrono::
99                 microseconds>(end - start).count();

```

```
95         total_comps += global_char_comparisons;
96     }
97
98     TestResult result;
99     result.avg_time_ms = (total_time_us / 1000.0) / runs;
100    result.avg_comparisons = total_comps / runs;
101
102    return result;
103 }
104 };
```

4 Посылки в codeforces

- **A1q** (STRING QUICK SORT): ID 375653083
- **A1m** (STRING MERGE SORT): ID 375654412
- **A1r** (MSD RADIX SORT): ID 375955355
- **A1rq** (MSD RADIX + QUICK SORT): ID 375955389

5 Публичный репозиторий

https://github.com/mvdolbilkin/set9_a1